

Progressive Feature Engineering and Imbalanced Machine Learning for Credit Card Fraud Detection

Author Name(s), Affiliation(s), Email Address(es)

Abstract

Credit card fraud detection is an imbalanced binary classification problem in which minority-class performance is more informative than overall accuracy. This paper presents a controlled tabular machine learning study using a researcher-generated synthetic credit card transaction dataset containing 499,985 unique records, including 62,500 fraudulent and 437,485 legitimate transactions. Five progressively enriched dataset versions are evaluated, ranging from baseline transaction attributes to engineered amount, time, merchant-risk, card-risk, and composite-risk features. Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost classifiers are trained under no-SMOTE and SMOTE settings. The experimental design uses a stratified 60/20/20 train-validation-test split, 5-fold cross-validation on the training split, PR-AUC as the hyperparameter tuning metric, validation-based model-family selection, and untouched test-set reporting. At the default threshold, SMOTE improves fraud-class F1 across all dataset versions; the strongest default-threshold result is obtained by Random Forest on v3 with SMOTE, reaching precision 0.998525, recall 0.758080, fraud-class F1 0.861846, and PR-AUC 0.855235. After validation-based threshold optimization, the practical difference between SMOTE and no-SMOTE is largely removed; one high-F1 tuned operating point is the v5 no-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 0.862030. The results show that progressive tabular feature engineering improves fraud-class performance, while conclusions are limited by the synthetic dataset, absence of external validation, and leakage-risk concerns around proxy risk indicators.

Keywords - credit card fraud detection; imbalanced classification; feature engineering; SMOTE; Random Forest; XGBoost; CatBoost; PR-AUC.

I. Introduction

A. Background and Motivation

Credit card fraud detection is a high-impact binary classification problem in which fraudulent transactions are less frequent than legitimate transactions but carry disproportionate operational risk. Classical fraud systems have relied on expert rules, threshold policies, and manual risk signals [1], [2]. Machine learning methods extend these approaches by learning statistical regularities from transaction-level data [3]-[8].

Because fraud detection is imbalanced, accuracy alone is not an adequate evaluation target. This manuscript therefore emphasizes fraud-class precision, recall, fraud-class F1, ROC-AUC, and PR-AUC, with average precision used for hyperparameter tuning [9]-[17].

B. Research Gap

Prior fraud detection studies often compare classifiers or resampling strategies, but fewer controlled repository-level studies isolate the effect of progressive feature enrichment while keeping the underlying transaction population fixed. This study evaluates five dataset versions built from the same cleaned master population so that performance differences can be attributed to feature availability rather than sample-size changes.

C. Aim and Contributions

The aim is to evaluate whether progressive tabular feature engineering improves fraud-class performance under a leakage-aware experimental protocol. The study contributes a reproducible five-version dataset preparation workflow, a no-SMOTE versus SMOTE comparison, validation-based model selection, untouched test-set reporting, and validation-based threshold optimization. The study does not evaluate language-processing features, institutional deployment, or external generalization.

D. Paper Organization

Section II reviews related work. Section III describes dataset construction and validation. Section IV presents the methodology. Section V reports the experimental results. Section VI discusses interpretation and reproducibility. Section VII states threats to validity, and Section VIII concludes the paper.

II. Related Work

A. Traditional Rule-Based Fraud Detection

Rule-based fraud systems encode expert knowledge through manually specified thresholds and transaction patterns. Such systems are interpretable but require maintenance when transaction behavior or attack strategies change [1], [2].

B. Machine Learning for Credit Card Fraud Detection

Machine learning methods have been widely applied to card fraud detection, including linear classifiers, decision trees, random forests, gradient boosting, and sequence models [3]–[8]. In this repository, the final active experiment includes Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost.

C. Imbalanced Classification and SMOTE

Fraud detection commonly presents class imbalance. SMOTE and related methods create synthetic minority-class examples to reduce majority-class dominance during learning [9]–[14]. This study applies SMOTE only inside the training pipeline after preprocessing and within cross-validation folds.

D. Feature Engineering for Tabular Fraud Detection

Feature engineering can transform transaction attributes into stronger predictors, including amount-derived variables, temporal indicators, merchant descriptors, and risk summaries [5], [29], [30]. The present study evaluates progressive feature groups from v1 to v5 while holding the master population constant.

E. Evaluation Metrics for Imbalanced Fraud Detection

PR-AUC and fraud-class F1 are emphasized because ROC-AUC and accuracy can overstate performance when the minority class is operationally important [15]–[17]. The repository uses average precision for tuning and validation-based fraud-class F1 for model-family selection.

III. Dataset Construction and Validation

A. Dataset Generation and Cleaning

The study uses a researcher-generated synthetic transaction dataset. The active final master file is `data/research_master_dataset.csv`. The raw merged provenance file contains 849,999 rows and 350,014

exact duplicate rows; it is not treated as a sixth experiment. After cleaning and transaction-level de-duplication, the final master contains 499,985 unique transactions.

TABLE I. Dataset Summary

Property	Value
Dataset file	data/research_master_dataset.csv
Dataset type	Researcher-generated synthetic transaction dataset
Task	Binary tabular classification
Target label	FraudFlag
Total transactions	499,985
Fraud transactions	62,500
Legitimate transactions	437,485
Fraud ratio	12.5004%
Missing values in final datasets	0
Exact duplicate rows in final datasets	0
Duplicate TransactionID values in final datasets	0
Language-processing features	Not present

B. Dataset Audit and Integrity Checks

The final datasets contain no missing values, no exact duplicate rows, and no duplicate TransactionID values. Transaction identifiers are deterministic synthetic hashes used for reproducibility and split-overlap checks. They are excluded from all model feature sets.

C. Class Distribution

The cleaned dataset contains 62,500 fraud transactions and 437,485 legitimate transactions, corresponding to a 12.5004% fraud ratio. Fig. 3 summarizes this distribution.

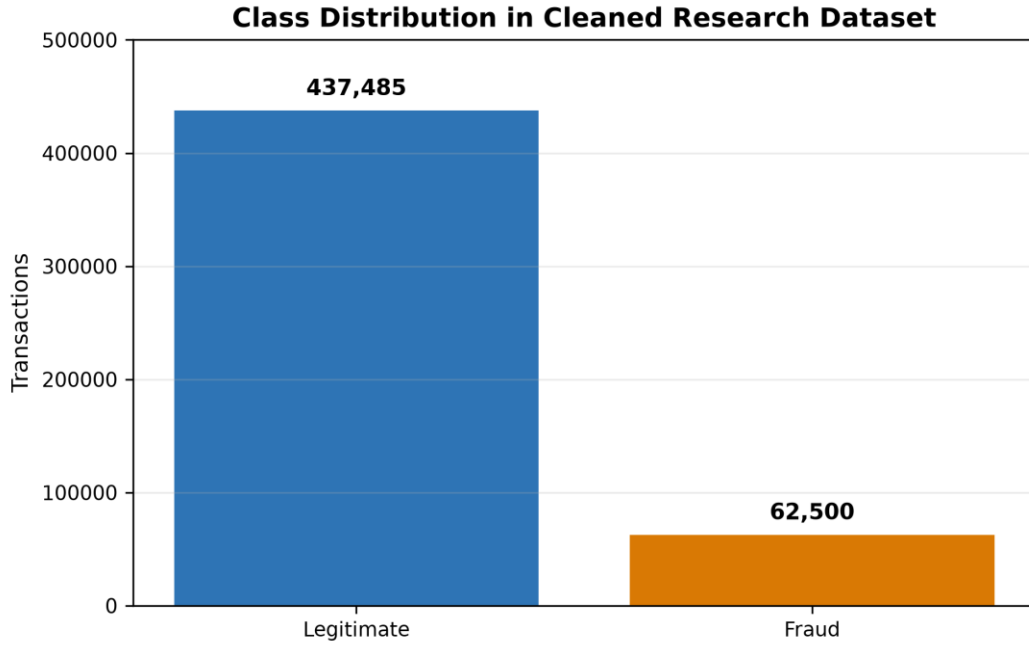


Fig. 3. Class distribution of the cleaned synthetic transaction dataset.

D. Progressive Dataset Versions

Five dataset versions are constructed from the same transaction population. Table II summarizes their feature additions, and Fig. 2 visualizes the progressive design.

TABLE II. Progressive Dataset Versions

Version	Rows	Columns	Added Feature Group	Purpose
v1	499,985	8	Baseline transaction, categorical, and time fields	Baseline comparator
v2	499,985	11	High-amount flag, night-transaction flag, subcategory	Assess simple risk indicators
v3	499,985	13	Merchant-risk and card-risk indicators	Assess proxy risk enrichment
v4	499,985	21	Amount and time engineered features	Assess advanced engineered predictors
v5	499,985	23	Night-high-amount flag and composite risk score	Assess full engineered feature set

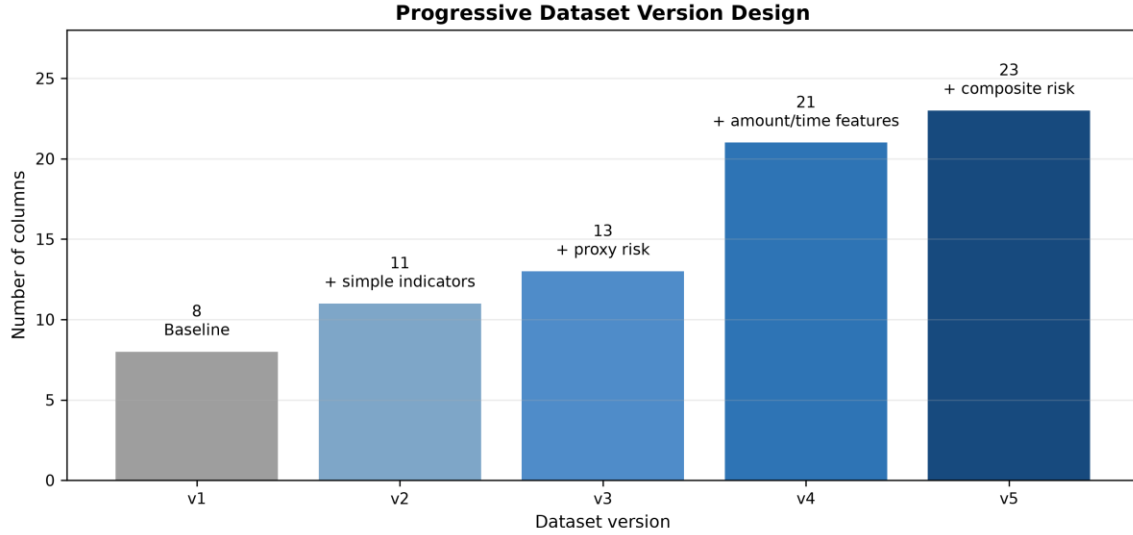


Fig. 2. Progressive dataset version design from v1 through v5.

E. Feature Exclusion and Leakage Controls

The active modeling pipeline excludes FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, and IsWeekendDerived from model features. Distribution-dependent engineered features are recomputed inside the modeling pipeline from training-fold data. MerchantRisk and CardRisk are treated as synthetic proxy risk indicators and are discussed as leakage-risk features rather than proven deployment-safe variables [25]-[28].

IV. Methodology

A. Experimental Pipeline

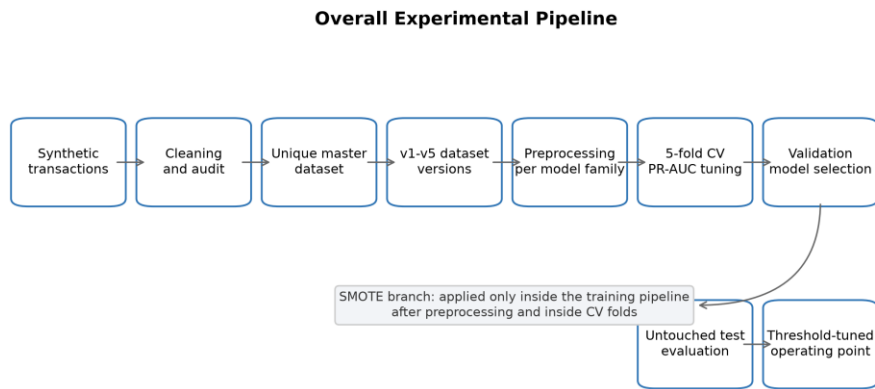


Fig. 1. Overall experimental pipeline for dataset construction, model tuning, validation selection, test evaluation, SMOTE comparison, and threshold optimization.

Fig. 1 summarizes the experimental workflow. The pipeline begins with synthetic dataset construction, cleaning, audit, and five-version dataset generation. Each version is then processed through model-specific

preprocessing, cross-validated hyperparameter tuning, validation-based selection, untouched test evaluation, and threshold optimization.

TABLE III. Experimental Configuration

Component	Configuration
Split strategy	Stratified train/validation/test split
Split ratio	60/20/20
Training records	299,991
Validation records	99,997
Test records	99,997
Cross-validation	5-fold CV on training split
Hyperparameter tuning metric	PR-AUC / average precision
Model-family selection	Validation split
Final reporting	Untouched test split
SMOTE placement	Inside imblearn pipeline, after preprocessing, inside CV folds
Excluded model features	FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, IsWeekendDerived

B. Train/Validation/Test Split

The experiment uses a stratified 60/20/20 split. The training split contains 299,991 records, the validation split contains 99,997 records, and the test split contains 99,997 records. Hyperparameter search is restricted to the training split, model-family selection is performed on the validation split, and final reporting uses the untouched test split.

C. Preprocessing Strategy

Logistic Regression uses imputation, one-hot encoding for categorical variables, and numeric scaling. Decision Tree, Random Forest, and XGBoost use imputation and frequency encoding for categorical variables. CatBoost follows the repository implementation: leakage-safe feature engineering is applied before CatBoost fitting and prediction, with categorical features handled through the final CatBoost workflow.

D. Model Selection and Hyperparameter Tuning

All model families are tuned with 5-fold cross-validation on the training split using PR-AUC, implemented as average precision. Candidate models are selected on validation fraud-class F1 with tie-breaking based on PR-AUC, recall, precision, ROC-AUC, and lower training time. Randomized hyperparameter search follows standard validation practice [23]-[26].

E. SMOTE Configuration

SMOTE is evaluated as a separate experimental condition. It is applied only inside the imblearn training pipeline, after preprocessing, and inside cross-validation folds. No SMOTE is applied before the train/validation/test split.

F. Threshold Optimization

Threshold optimization is performed after training by selecting a fraud-probability threshold on validation predictions and evaluating that selected threshold on test predictions. This provides an operating-point analysis distinct from default-threshold model comparison.

G. Evaluation Metrics

The study reports fraud-class precision, fraud-class recall, fraud-class F1, ROC-AUC, and PR-AUC. Accuracy is not used as the primary selection criterion because the minority class is the operational focus.

V. Experimental Results

A. Default-Threshold No-SMOTE Results

Table IV reports the best default-threshold no-SMOTE model for each dataset version. Performance increases sharply from v1-v2 to v3-v5, indicating that the risk-enriched and engineered versions provide more useful tabular predictors.

TABLE IV. Best Default-Threshold Results Without SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	XGBoost	0.8062	0.7502	0.7772	0.8268
v2	XGBoost	0.8094	0.7500	0.7786	0.8274
v3	CatBoost	0.9219	0.7739	0.8414	0.8558
v4	DecisionTree	0.9537	0.7678	0.8507	0.8337
v5	RandomForest	0.9418	0.7713	0.8480	0.8561

B. Default-Threshold SMOTE Results

Table V reports the best default-threshold SMOTE model for each dataset version. SMOTE improves fraud-class F1 at the default threshold for all five versions. The best default-threshold result is v3 SMOTE Random Forest with fraud-class F1 of 0.861846 and PR-AUC of 0.855235.

TABLE V. Best Default-Threshold Results With SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	DecisionTree	0.9760	0.7040	0.8180	0.8237
v2	RandomForest	0.9527	0.7153	0.8171	0.8265
v3	RandomForest	0.9985	0.7581	0.8618	0.8552
v4	CatBoost	0.9998	0.7570	0.8616	0.8564
v5	CatBoost	0.9999	0.7571	0.8617	0.8558

C. Feature Engineering Impact

Fig. 4 shows the fraud-class F1 comparison across dataset versions. The strongest performance jump occurs when moving from v2 to v3. Versions v3, v4, and v5 are close, so the evidence supports practical similarity among the strongest feature sets rather than a claim that any one of these versions is materially dominant.



Fig. 4. Fraud-class F1 comparison between no-SMOTE and SMOTE selected models across dataset versions.

D. SMOTE Impact

SMOTE improves default-threshold fraud-class F1 but does not universally improve recall. Fig. 5 and Fig. 6 show that the SMOTE setting often increases precision while reducing recall for the selected default-threshold models. Fig. 7 summarizes the PR-AUC comparison.



Fig. 5. Fraud precision comparison between no-SMOTE and SMOTE selected models.



Fig. 6. Fraud recall comparison between no-SMOTE and SMOTE selected models.

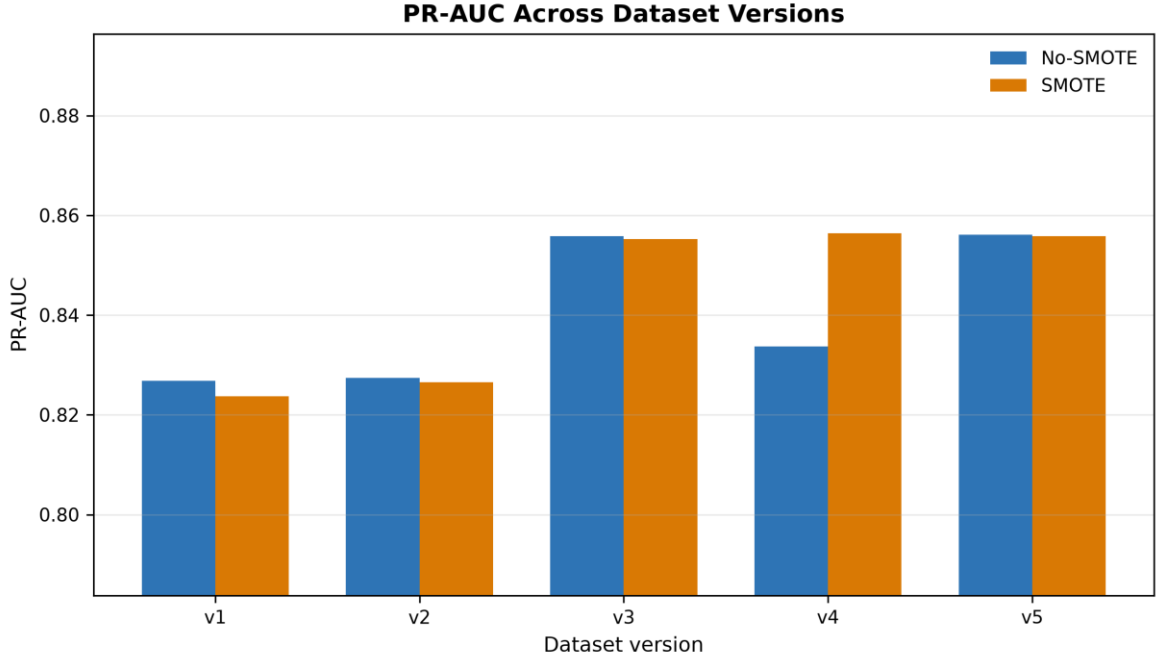


Fig. 7. PR-AUC comparison between no-SMOTE and SMOTE selected models.

E. Threshold-Tuned Results

Table VI reports validation-based threshold-tuned operating points. Threshold tuning largely removes the practical advantage of SMOTE. The v5 no-SMOTE Random Forest operating point at threshold 0.80 records precision 0.999156, recall 0.758000, and fraud-class F1 0.862030.

TABLE VI. Threshold-Tuned Comparison

Version	No-SMOTE Model	No-SMOTE Threshold	No-SMOTE F1	SMOTE Model	SMOTE Threshold	SMOTE F1
v1	RandomForest	0.80	0.819141	CatBoost	0.44	0.818983
v2	XGBoost	0.83	0.818759	XGBoost	0.43	0.818828
v3	XGBoost	0.82	0.861660	XGBoost	0.51	0.861611
v4	DecisionTree	0.76	0.861826	XGBoost	0.47	0.861443
v5	RandomForest	0.80	0.862030	XGBoost	0.45	0.861520

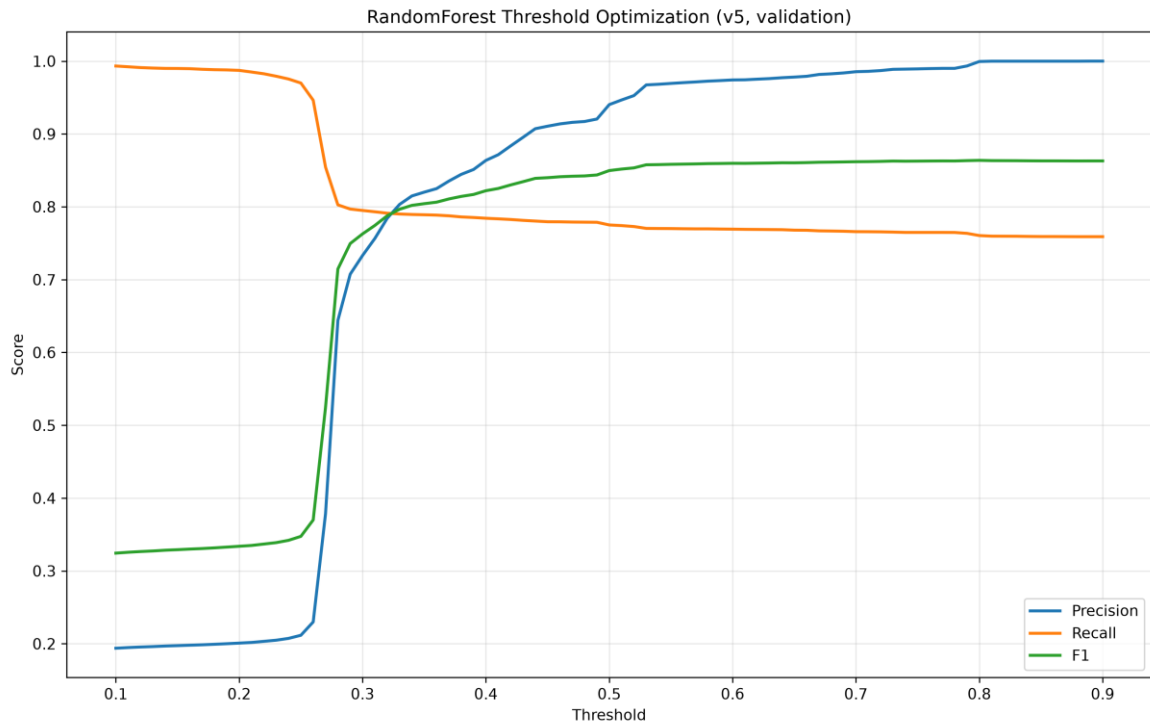


Fig. 8. Threshold optimization curve for the v5 no-SMOTE Random Forest model.

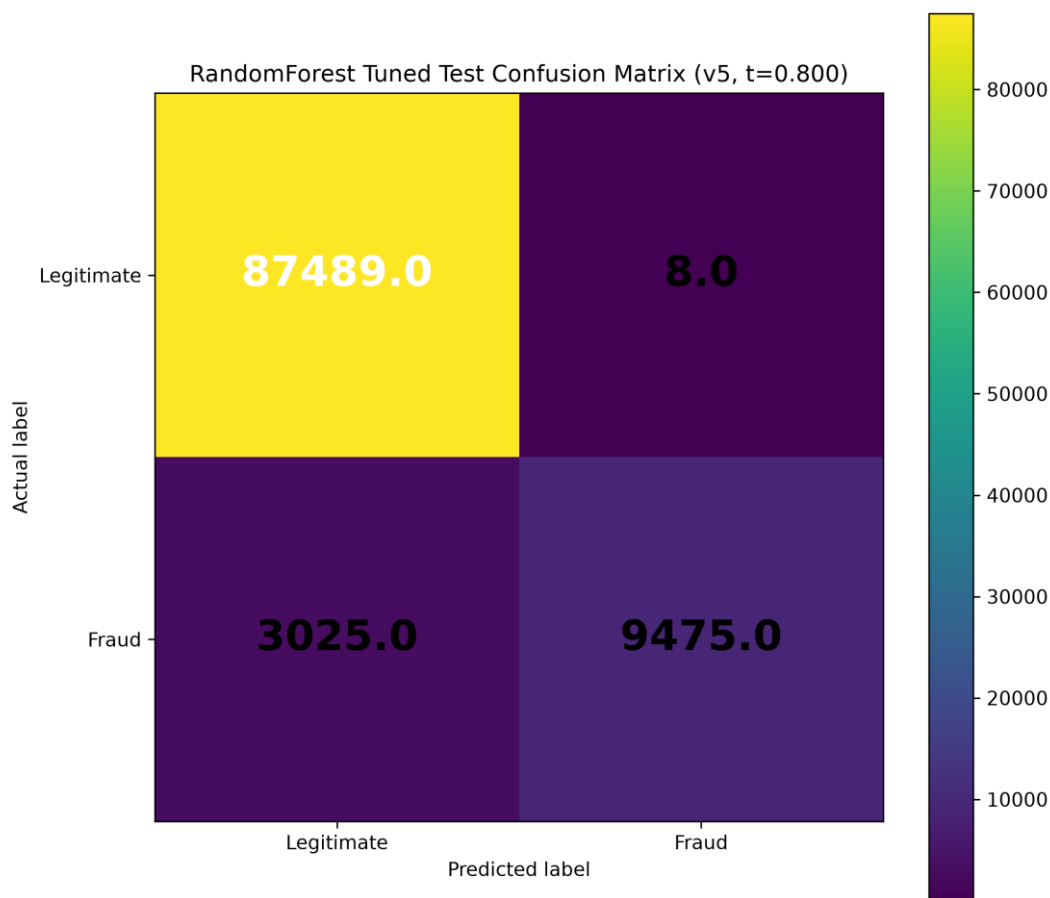


Fig. 9. Test-set confusion matrix for the v5 no-SMOTE Random Forest model at threshold 0.80.

F. Best Model Analysis

The strongest default-threshold and threshold-tuned results are achieved by tree-based models. Fig. 10 reports impurity-based feature importance for the selected v5 no-SMOTE Random Forest model. These importance values are model-specific and should not be interpreted as causal effects.

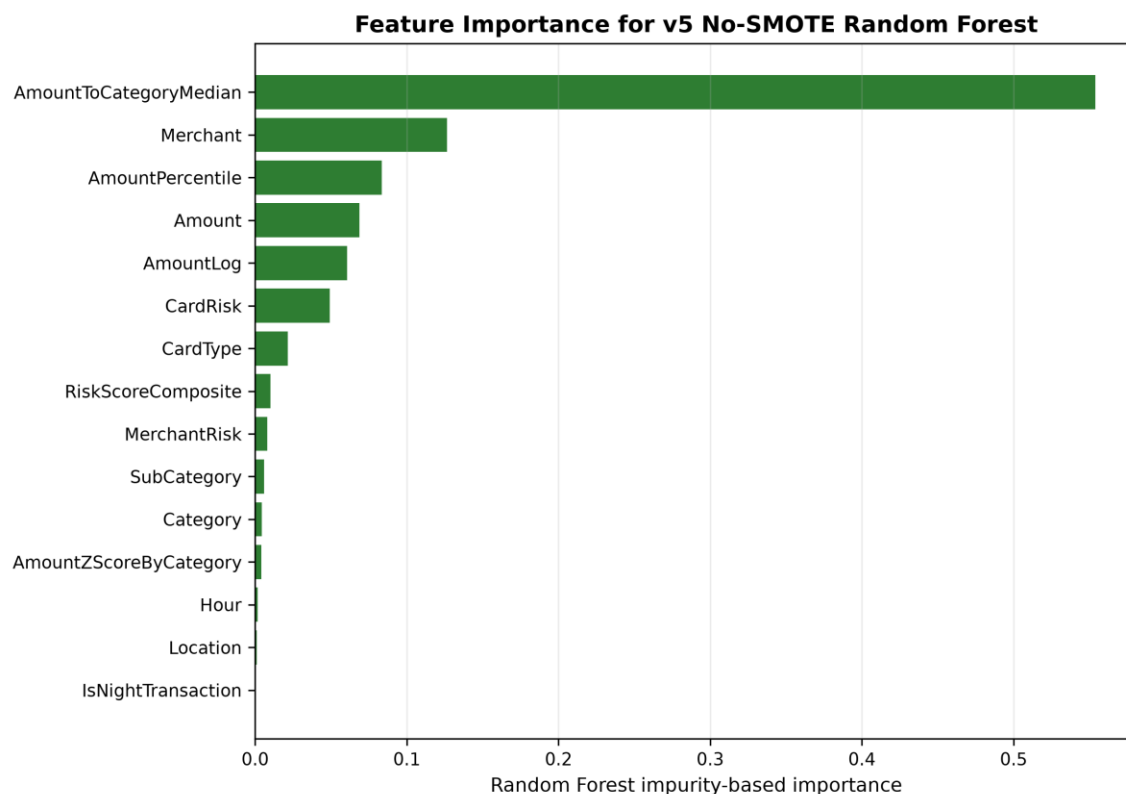


Fig. 10. Impurity-based feature importance for the selected v5 no-SMOTE Random Forest model.

TABLE VII. Summary of Main Findings

Finding	Evidence	Interpretation
Progressive feature engineering improves fraud-class performance	v3-v5 outperform v1-v2 in fraud-class F1	Feature enrichment changes minority-class separability.
SMOTE improves default-threshold fraud-class F1	SMOTE selected models outperform no-SMOTE selected models in all five versions	The benefit is threshold-dependent.
Threshold tuning removes most practical SMOTE advantage	Best tuned F1 values differ by less than 0.001 across setups	Operating-point selection is central to deployment interpretation.
Tree-based methods dominate	Random Forest, XGBoost, CatBoost, and Decision Tree are selected in final best-model summaries	Nonlinear tabular interactions are important in this dataset.
Logistic Regression underperforms	No final best model is Logistic Regression	Linear baseline is less competitive for the engineered feature space.
Strongest versions are practically close	v3-v5 default and tuned results are close	Do not overstate the small differences among v3-v5.
External validity is limited	No external validation dataset is present	Results apply to the synthetic transaction dataset only.

VI. Discussion

A. Interpretation of Main Findings

The results support the claim that progressive tabular feature engineering improves fraud-class performance on the synthetic transaction dataset. The improvement is most evident when moving from v1-v2 to v3-v5. The strongest versions are practically close, and the manuscript therefore interprets differences among v3-v5 conservatively.

B. Why Tree-Based Models Performed Better

Tree-based methods are well suited to heterogeneous tabular features because they can model nonlinear thresholds and interactions among amount, category, location, transaction timing, and proxy risk variables [18]-[22]. Logistic Regression is retained as a baseline but is not selected as a best model in the final results.

C. SMOTE Versus Threshold Optimization

The default-threshold results suggest that SMOTE improves fraud-class F1. The threshold-tuned results show a more cautious interpretation: once operating thresholds are selected on validation predictions, no-SMOTE and SMOTE settings become practically similar. This indicates that resampling conclusions should be reported with the associated threshold policy.

D. Practical Implications

For fraud analysis workflows, the results emphasize the importance of feature design, validation-based threshold selection, and minority-class metrics. The highest-F1 tuned operating point is highly precise but has moderate recall, meaning that a different threshold could be preferred if the objective were to capture more fraudulent transactions at the cost of additional false positives.

E. Reproducibility Considerations

The repository provides deterministic dataset files, final training outputs, validation and test prediction files, threshold-tuning outputs, and comparison summaries. The final manuscript uses only the active research-clean pipeline and result folders, not stale README instructions or legacy result folders.

VII. Threats to Validity

A. Synthetic Dataset Limitation

The dataset is researcher-generated and synthetic. The repository does not document an institutional financial-data source. Therefore, the results should not be interpreted as evidence of performance on operational card transaction streams.

B. External Validity Limitation

No external validation dataset is present. All results are computed from splits of the same cleaned synthetic master dataset. Cross-institution, temporal, or public-benchmark validation remains future work.

C. Leakage-Risk Features

MerchantRisk and CardRisk are proxy risk indicators and may not be leakage-free. Their availability and construction at prediction time must be validated before these features can be considered deployable.

D. Threshold Dependence

The effect of SMOTE depends on the classification threshold. Default-threshold F1 improves under SMOTE, but validation-tuned operating points are practically tied across no-SMOTE and SMOTE settings.

E. Absence of Confidence Intervals

The repository does not contain repeated-seed experiments, confidence intervals, bootstrap uncertainty estimates, or formal significance tests. Small differences among v3-v5 should therefore be interpreted conservatively.

F. Documentation Drift and Legacy Scripts

The repository contains legacy scripts and stale documentation that do not represent the final research-clean workflow. This manuscript cites only the active final scripts, datasets, and result folders.

VIII. Conclusion and Future Work

This paper evaluated progressive feature engineering and imbalanced learning for credit card fraud detection using a synthetic transaction dataset. Five dataset versions and five model families were evaluated under no-SMOTE and SMOTE settings with stratified splitting, cross-validated PR-AUC tuning, validation-based model selection, and untouched test reporting.

The results show that v3-v5 substantially outperform v1-v2, that tree-based models dominate the strongest results, and that SMOTE improves default-threshold fraud-class F1. However, threshold tuning largely removes the practical advantage of SMOTE. The highest default-threshold fraud-class F1 is 0.861846 for v3 SMOTE Random Forest, while the highest threshold-tuned fraud-class F1 is 0.862030 for v5 no-SMOTE Random Forest.

Future work should validate the pipeline on external transaction data, document all feature provenance, test the robustness of MerchantRisk and CardRisk, estimate uncertainty across repeated runs, and evaluate operating thresholds under explicit false-positive and false-negative cost assumptions.

References

- [1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235-255, 2002, doi: 10.1214/ss/1042727940.
- [2] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," *Decision Support Systems*, vol. 50, no. 3, pp. 559-569, 2011, doi: 10.1016/j.dss.2010.08.006.
- [3] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, pp. 602-613, 2011, doi: 10.1016/j.dss.2010.08.008.
- [4] C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, and N. M. Adams, "Transaction aggregation as a strategy for credit card fraud detection," *Data Mining and Knowledge Discovery*, vol. 18, no. 1, pp. 30-55, 2009, doi: 10.1007/s10618-008-0116-z.
- [5] A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," *Expert Systems with Applications*, vol. 51, pp. 134-142, 2016, doi: 10.1016/j.eswa.2015.12.030.
- [6] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symposium Series on Computational Intelligence*, 2015.
- [7] J. Jurgovsky et al., "Sequence classification for credit-card fraud detection," *Expert Systems with Applications*, vol. 100, pp. 234-245, 2018, doi: 10.1016/j.eswa.2018.01.037.
- [8] C. Phua, V. Lee, K. Smith, and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," arXiv:1009.6119, 2010.

- [9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.
- [10] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263-1284, 2009, doi: 10.1109/TKDE.2008.239.
- [11] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intelligent Data Analysis*, vol. 6, no. 5, pp. 429-449, 2002.
- [12] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 42, no. 4, pp. 463-484, 2012, doi: 10.1109/TSMCC.2011.2161285.
- [13] P. Branco, L. Torgo, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," *ACM Computing Surveys*, vol. 49, no. 2, pp. 1-50, 2016, doi: 10.1145/2907070.
- [14] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. International Joint Conference on Artificial Intelligence*, 2001, pp. 973-978.
- [15] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proc. International Conference on Machine Learning*, 2006, pp. 233-240, doi: 10.1145/1143844.1143874.
- [16] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, 2015, doi: 10.1371/journal.pone.0118432.
- [17] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861-874, 2006, doi: 10.1016/j.patrec.2005.10.010.
- [18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.
- [19] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81-106, 1986, doi: 10.1007/BF00116251.
- [20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.
- [21] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorigush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.
- [22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [23] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281-305, 2012.
- [24] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. International Joint Conference on Artificial Intelligence*, 1995, pp. 1137-1145.
- [25] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079-2107, 2010.
- [26] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, article 91, 2006, doi: 10.1186/1471-2105-7-91.
- [27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1-21, 2012, doi: 10.1145/2382577.2382579.
- [28] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023, doi: 10.1016/j.patter.2023.100804.
- [29] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157-1182, 2003.
- [30] M. Kuhn and K. Johnson, *Applied Predictive Modeling*. New York, NY, USA: Springer, 2013.